

RAG (retrieval-augmented-generation)

1. why do we need RAG?

1. give LLM knowledge it didn't learn
eg. model trained in 2024 → what happened in 2026?

2. dealing with long context

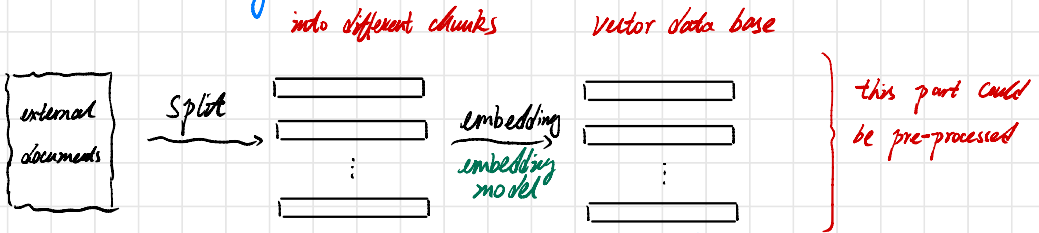
We can't always just paste relevant information into prompt

1. limited context window
2. latency issue $O(n^2)$ complexity
3. memory issue (model focuses ^{beginning} → end)

3. data security

1. keep data internal
2. machine unlearning is hard (if we use RAG, simply delete)

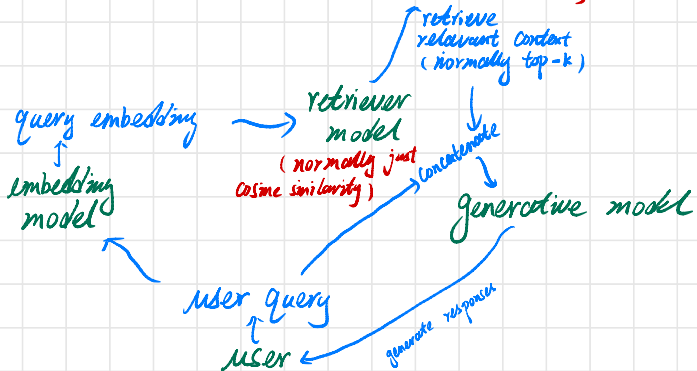
2. how does RAG actually work?



note: 1. embedding model for users query and text are the same.

2. retriever model is normally just cosine similarity, but could be more complex, eg. Combined with key word search or using K-MNN etc.

3. where query and context are normally concatenated as follows:



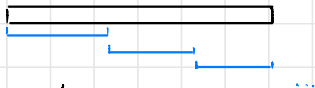
System: [system prompt]

Context: [retrieved context]

User: [user query]

Split

1. naive method:



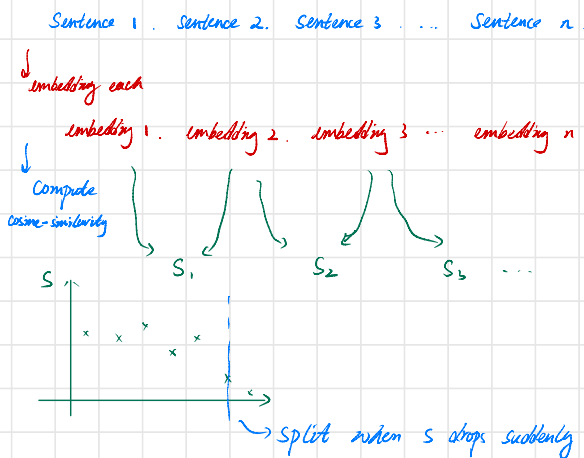
1. fix-sized sliding window

problem: break the logic chain

2. Split by structure, e.g. by paragraphs or sentences.

problem: several different paragraphs could all be relevant.

3. Semantic chunking



4. Late chunking



important: now the embedding for each token is its "global meaning"

Embedding & Retrieval dilemma: Latency & memory issue vs quality

↓

We want to use sparse or less bits to represent a vector, but this lowers the quality

↓

We want high quality retrieval.

problem: high dimensional vectors capture good semantic nuance, but they are expensive to store and have high latency.

Solution: **Matrix Factorization Representation Learning**



first 8, 16, 32, 64, ... 1024 dimensions will simultaneously do the pairing, and the loss will be the average sum of these losses

result: information is dense in the first a few dimensions.

problem: how to search efficiently among millions of vector database

Solution: hierarchical navigable small world

Say we have $v_1, \dots, v_n$, where n is large

We construct a graph (here we take $n=10$ as an example)

